

Unit 8- Collaborative Discussion 4 – Refactoring and Code Smells

Initial Post

The supplied `calculate_total_price()` function works, but two clear code smells reduce its maintainability. First, the discount factors 0.9 and 0.8 are magic numbers. Their meaning is not obvious from the calculation itself, and changing a discount requires editing the function. Replacing them with named constants would make the pricing rules clearer and easier to maintain (Bergmann, 2026).

Second, the function contains an expanding `if/elif/else` conditional that selects behaviour according to `item["type"]`. As more product types are introduced, additional branches will be added to the same function, increasing complexity and making unrelated pricing rules harder to separate.

For a small and stable system, I would begin with the simpler refactoring:

```
BOOK_DISCOUNT = 0.9
ELECTRONICS_DISCOUNT = 0.8

def calculate_total_price(items):
    total = 0

    for item in items:
        if item["type"] == "book":
            total += item["price"] * BOOK_DISCOUNT
```

```

elif item["type"] == "electronics":

    total += item["price"] * ELECTRONICS_DISCOUNT

else:

    total += item["price"]

return total

```

If discount behaviours are expected to grow or become more complex, the Strategy Pattern would be more scalable because each discount rule can be separated behind a common interface (Refactoring.Guru, n.d.). The appropriate refactoring approach depends on the complexity of the system and how likely the pricing rules are to change. In the scalable version, each discount is implemented as a separate strategy, while a strategy mapping replaces the original conditional selection in `calculate_total_price()`.

Scalable Alternative – Strategy Pattern

```

from abc import ABC, abstractmethod

class DiscountStrategy(ABC):

    @abstractmethod
    def apply(self, price):
        pass

class BookDiscount(DiscountStrategy):
    DISCOUNT_FACTOR = 0.9

    def apply(self, price):
        # Apply a 10% discount to book items
        return price * self.DISCOUNT_FACTOR

```

```
class ElectronicsDiscount(DiscountStrategy):
    DISCOUNT_FACTOR = 0.8

    def apply(self, price):
        # Apply a 20% discount to electronics items
        return price * self.DISCOUNT_FACTOR

class NoDiscount(DiscountStrategy):

    def apply(self, price):
        # Return the original price
        return price

# Create one default strategy
NO_DISCOUNT = NoDiscount()

# Map each product type to its corresponding discount strategy
DISCOUNT_STRATEGIES = {
    "book": BookDiscount(),
    "electronics": ElectronicsDiscount()
}

def calculate_total_price(items):
    total = 0

    for item in items:
        # If the item type is not found, use NoDiscount
        strategy = DISCOUNT_STRATEGIES.get(
            item["type"],
            NO_DISCOUNT
        )

        total += strategy.apply(
            item["price"]
        )

    return total
```

Reference

Refactoring.Guru (n.d.) *Strategy*. Available at: <https://refactoring.guru/design-patterns/strategy>

Bergmann, D. (2026) *Code smells, explained*. Available at: <https://www.ibm.com/think/topics/code-smells>